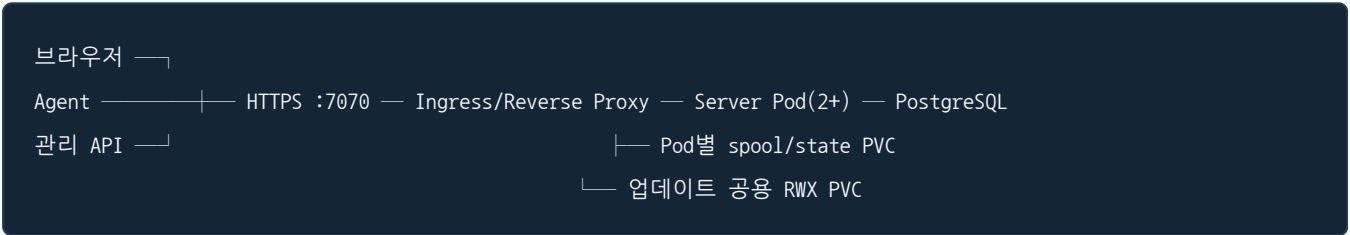


Invenqor Server 설치·운영·오프라인 배포 가이드

대상 버전: v0.2.1 · 기준일: 2026-07-29

1. 운영 구조와 단일 포트

Invenqor는 Linux Agent, 중앙 Server, PostgreSQL, 웹 관리 콘솔로 구성됩니다. 사용자 UI, 관리자 API, Agent 수집·heartbeat·업데이트 트래픽은 모두 Server의 기본 TCP 7070 한 포트를 공유합니다. 방화벽에는 외부 공개 포트를 여러 개 추가하지 않아도 됩니다.



운영망에서는 7070 에서 TLS를 종료하는 Ingress 또는 Reverse Proxy를 사용하고 Agent의 server.url 도 같은 URL을 지정합니다. Agent는 inbound 포트를 열지 않으며 Server로 outbound 연결만 생성합니다.

2. 무엇을 수집하고 어떻게 저장하는가

영역	주요 항목	기본 원천
시스템	호스트명, 배포판, Kernel, Architecture, Boot time, Timezone	/etc/os-release , /proc
CPU·메모리	모델, 논리 CPU, load, 메모리·swap 지표	/proc/cpuinfo , /proc/meminfo
파일시스템	장치, Mount, 유형, 옵션, 용량, inode	/proc/self/mounts , statvfs
네트워크	Interface, MAC/IP, MTU, 상태, Route, DNS, 로컬 포트	/sys/class/net , /proc/net
프로세스	PID/PPID, 이름, 상태, UID/GID, 실행 파일	/proc/[pid]
패키지	dpkg/rpm/apk 이름, 버전, Architecture	배포판 패키지 DB 또는 고정 rpm -qa
서비스	systemd/OpenRC/SysV 상태와 시작 정책	각 init 조회 인터페이스
계정	사용자·그룹, UID/GID, 홈, Shell, 보조 그룹	/etc/passwd , /etc/group
컨테이너	Runtime socket, cgroup, 컨테이너 내부 여부	Runtime·cgroup 표식

프로세스 명령행은 비밀정보가 들어갈 수 있어 기본 미수집입니다. 파일 본문, 비밀번호 해시, 환경변수, 키·토큰, 패킷 내용은 수집하지 않습니다.

Server는 `Agent UUID + category + asset_id`를 원천 키로 보관합니다. 첫 전송은 전체 Snapshot, 이후에는 `added/updated/removed` 변경분만 전송합니다. Collector 오류가 발생한 주기에는 누락을 삭제로 오판하지 않습니다. 원본 이벤트, 현재 Snapshot, 변경 이력과 오류를 함께 보존해 화면 값의 출처를 추적할 수 있습니다.

3. 온라인 Docker Compose 설치

필수 조건은 Docker Engine 24+, Compose v2, 4 GiB 이상의 여유 메모리입니다.

```
git clone https://github.com/hkjang/invenqor.git
cd invenqor
export POSTGRES_PASSWORD="$(openssl rand -base64 32)"
docker compose up -d --build
curl -fsS http://127.0.0.1:7070/health/ready
```

`http://서버:7070`에서 콘솔을 엽니다. 운영 전에는 TLS Proxy, DNS, PostgreSQL 백업, 시간 동기화와 로그 수집을 구성하십시오.

4. 폐쇄망·오프라인 설치

GitHub Release의 두 파일을 인터넷 연결 구간에서 내려받아 승인된 매체로 반입합니다.

- `invenqor-0.2.1.tar.gz`
- `invenqor-0.2.1.tar.gz.sha256`
- 함께 제공되는 `compose.offline.yaml`

무결성 검증 후 Docker에 Server와 PostgreSQL 이미지를 한 번에 적재합니다.

```
sha256sum -c invenqor-0.2.1.tar.gz.sha256
gzip -t invenqor-0.2.1.tar.gz
docker load < invenqor-0.2.1.tar.gz
docker image inspect invenqor-server:0.2.1 --format '{{.Id}} {{.Architecture}}'
docker image inspect postgres:17-alpine --format '{{.Id}} {{.Architecture}}'
```

`compose.offline.yaml`은 `pull_policy: never`이므로 외부 Registry를 조회하지 않습니다.

```
export POSTGRES_PASSWORD="$(openssl rand -base64 32)"
docker compose -f compose.offline.yaml up -d
curl -fsS http://127.0.0.1:7070/health/ready
```

반입 이미지는 `linux/amd64`용입니다. ARM 서버에는 `x86_64` 이미지를 실행하지 마십시오. 제공된 빌드 스크립트는 이미지 두 개를 `docker save` 호환 gzip으로 만들고 SHA-256 파일까지 생성합니다.

```
./scripts/build-offline-images.sh 0.2.1
```

5. 최초 관리자와 Agent 등록

Server는 최초 기동 시 일회용 관리자 토큰을 상태 볼륨에 만듭니다.

```
docker compose exec server \
  cat /var/lib/invenqor-server/initial-admin.token
```

```
curl -X POST http://127.0.0.1:7070/api/v1/bootstrap/admin \
  -H "X-Invenqor-Bootstrap-Token: $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin","password":"CorrectHorse!42","display_name":"관리자"}'
```

성공 즉시 토큰은 DB에서 폐기되어 재사용할 수 없습니다. 멀티 파드에서는 한 Pod만 토큰 파일을 생성하지만 어느 Pod로 요청해도 같은 DB 해시로 검증됩니다.

관리 콘솔에서 Agent UUID를 등록하고 한 번만 노출되는 장비별 Bearer Token을 Agent 설정에 넣습니다. 서버 DB에는 Token 원문이 아닌 SHA-256 해시만 저장됩니다.

```
[server]
url = "https://invenqor.example.com:7070"
bearer_token = "ivq_at_..."
ca_file = "/etc/invenqor-agent/ca.pem"
allow_insecure_http = false
timeout_seconds = 30
```

`allow_insecure_http=true` 는 격리된 E2E망 전용입니다. 운영 설정에서는 HTTPS를 유지하십시오.

6. Agent 설치와 실제 기동

Release의 CPU별 정적 musl 패키지를 사용합니다. 이 방식은 CentOS 7처럼 오래된 glibc가 있는 호스트에도 별도 런타임을 요구하지 않습니다.

```

curl -fL0 https://github.com/hkjang/invenqor/releases/download/v0.2.1/invenqor-agent-linux-x86_64.tar.gz
curl -fL0 https://github.com/hkjang/invenqor/releases/download/v0.2.1/invenqor-agent-linux-x86_64.tar.gz.sha256
sha256sum -c invenqor-agent-linux-x86_64.tar.gz.sha256
tar -xzf invenqor-agent-linux-x86_64.tar.gz
sudo ./invenqor-agent-linux-x86_64/scripts/install.sh
sudoedit /etc/invenqor-agent/config.toml
sudo /opt/invenqor-agent/bin/invenqor-agent --validate-config
sudo systemctl restart invenqor-agent
sudo systemctl status invenqor-agent --no-pager

```

SysV와 OpenRC 정의도 패키지에 포함됩니다. 상태 디렉터리와 `agent-id` 는 재설치·업데이트 중 보존해야 같은 장비로 계속 인식됩니다.

7. 관리 부담을 줄이는 자동 동작

- Server는 시작 시 DB Schema를 자동 마이그레이션합니다.
- PostgreSQL 멀티 파드 마이그레이션은 advisory lock으로 한 Pod씩 수행됩니다.
- 일시적 DB 장애에는 인증된 Agent 이벤트를 Pod별 append-only spool에 쓰고 DB 복구 뒤 자동 재처리합니다.
- Agent는 변경분만 보내고, 전송 실패 시 내구성 큐와 지수 backoff로 자동 재시도합니다.
- 장비별 Token은 관리 API에서 유효기간을 둔 회전이 가능하며 차단은 즉시 모든 Pod에 적용됩니다.
- 서명된 업데이트는 Agent가 주기적으로 확인·검증·스테이징하고 systemd path unit이 root helper를 한 번만 호출해 원자 교체합니다.

사람이 반드시 수행할 작업은 최초 관리자 생성, Agent 최초 등록, TLS/서명 개인키 보관, 백업 복구 훈련과 업데이트 승인입니다.

8. 서명된 Agent 자동 업데이트

업데이트 전용 Ed25519 개인키는 Server에 두지 말고 오프라인 서명 환경에 보관합니다. Agent에는 32-byte 공개키의 base64만 고정합니다.

```

openssl genpkey -algorithm ED25519 -out update-private.pem
openssl pkey -in update-private.pem -pubout -outform DER |
  tail -c 32 | base64 | tr -d '\n'
openssl pkeyutl -sign -rawin -inkey update-private.pem \
  -in invenqor-agent -out invenqor-agent.sig
base64 < invenqor-agent.sig | tr -d '\n'

```

Agent 설정:

```
[updates]
enabled = true
channel = "stable"
check_interval_seconds = 21600
public_key = "위에서-구한-base64-공개키"
install_path = "/opt/invenqor-agent/bin/invenqor-agent"
```

관리자는 `agents.manage` 권한과 CSRF Token으로 artifact, version, channel, OS, architecture, signature, rollout percentage를 `POST /api/v1/admin/agent-updates` 에 multipart로 게시합니다. Agent는 자신보다 높은 버전만 받고, 인증된 단일 `7070` 연결로 다운로드합니다. 최대 128 MiB, SHA-256, 크기, OS/Architecture와 Ed25519 서명을 모두 확인한 뒤에만 `updates/pending.json` 을 만듭니다. 적용 시 기존 바이너리는 `.previous` 로 보존되고 원자적 rename 실패 시 즉시 복원됩니다.

Canary는 `rollout_percent` 를 1~10으로 시작하고 중앙 수신·오류율을 확인한 뒤 단계적으로 100까지 올리십시오. 개인키 유출 시 즉시 배포 중단, 공개키 교체, 새 패키지 배포를 수행합니다.

9. Kubernetes 멀티 파드

필수 조건은 외부 PostgreSQL, 모든 Pod가 공유하는 32-byte Master Key Secret, Pod별 RWO state PVC, 업데이트용 RWX PVC입니다.

```
head -c 32 /dev/urandom > master.key
kubectl create secret generic invenqor-master-key --from-file=master.key
kubectl create secret generic invenqor-database \
  --from-literal=dsn='postgres://user:password@host/db?sslmode=require'
helm upgrade --install invenqor deploy/helm/invenqor \
  --set replicaCount=2 \
  --set updates.storageClassName='YOUR-RWX-STORAGE-CLASS'
```

Chart는 StatefulSet parallel 기동, readiness/liveness/startup probe, Pod anti-affinity, rolling update와 PDB `minAvailable: 1` 을 포함합니다. 세션, RBAC, Agent 상태와 감사 로그는 PostgreSQL에 있으므로 어느 Pod로 접속해도 동일합니다. Master Key가 Pod마다 다르다면 암호화된 OIDC/TOTP 비밀을 해독할 수 없으므로 Secret을 교체하거나 분실하지 마십시오. RWX StorageClass가 없는 클러스터는 업데이트 공유 저장소를 별도 Object Storage 구현으로 대체하기 전까지 replica 1로 운영해야 합니다.

10. 상태 확인, 백업과 복구

경로	정상 기준
<code>/health/live</code>	HTTP 200, 프로세스 생존
<code>/health/ready</code>	HTTP 200, 요청 처리 준비

경로	정상 기준
<code>/health/database</code>	<code>POSTGRES_ACTIVE</code> 권장
<code>/api/v1/system/info</code>	버전 <code>0.2.1</code> , 포트 <code>7070</code> , DB 모드

백업 대상은 PostgreSQL, Pod별 state/spool PVC, 업데이트 RWX PVC와 Master Key Secret입니다. DB와 Master Key는 같은 복구 시점으로 보호하십시오. 복구 훈련은 별도 Namespace에서 로그인, Agent 재전송, 감사 로그와 update manifest까지 확인해야 완료입니다.

11. 검증된 호환성

v0.2.1 E2E는 실제 PostgreSQL-backed Server와 Agent 컨테이너를 기동하고 수집 레코드 생성, 인증 전송, DB 처리, daemon 지속 실행과 서명 업데이트 스테이징을 확인했습니다.

OS 이미지	결과
Alpine	통과
CentOS 7	통과
Red Hat UBI 8	통과
Red Hat UBI 9	통과
Ubuntu 22.04 LTS	통과
Ubuntu 24.04 LTS	통과

별도 E2E에서 Server 두 Pod를 동시에 시작해 migration, readiness, 교차 Pod 로그인 세션을 확인했습니다. 재현 명령은 `./scripts/e2e-client-server.sh` 와 `./scripts/e2e-multipod.sh` 입니다.

12. 장애 판단

- `401` : Agent UUID와 장비별 Token, 차단 여부를 확인합니다.
- TLS 오류: URL hostname, 사설 CA, 인증서 만료와 7070 경로를 확인합니다.
- 큐 증가: Server readiness와 네트워크를 복구하십시오. 큐를 임의 삭제하지 않습니다.
- `SQLITE_FALLBACK` : PostgreSQL DSN/DNS/TLS/인증을 수정하고 운영 전 PostgreSQL 모드로 재기동합니다.
- 업데이트 미적용: 공개키, signature, SHA-256, version, architecture, rollout bucket과 systemd update path unit을 확인합니다.
- 멀티 파드 일부 실패: 공통 Master Key, RWX 권한, PostgreSQL advisory lock, 해당 Pod의 state/spool PVC를 확인합니다.